

Protein Network Discovery From AML Proteomics Data

Aaron Cui

The objective of this task was to infer biologically meaningful protein–protein interaction (PPI) structure from quantitative proteomics profiles of acute myeloid leukemia (AML) patient samples.

Data

The initial challenge involved managing missing data within a protein-by-patient matrix comprising 12,241 proteins across 118 samples. Proteins entirely absent across all patient samples were removed during preprocessing (12,142 proteins remained). Crucially, missing values were not replaced with zeros. Mathematically, substituting a zero into $\log_2$ -transformed data incorrectly implies a linear ratio of 1 ($2^0 = 1$), rather than a true absence. Biologically, non-detection often signifies low abundance or complete biological absence. To preserve these insights during network construction, the pipeline utilized methods that either computed similarity strictly across jointly observed patients or explicitly modeled protein detection patterns.

Methodological Choices

Stage 1: Pairwise Protein Similarity Estimation

The central hypothesis of this project is that proteins exhibiting similar abundance profiles or coordinated detection patterns across patients are more likely to interact. For comprehensively capturing abundance co-variation, non-linear profile similarity, and the structural nuances of missing data, I tried a couple of methods listed in Table 1.

Type	Method	Purpose	Output
Pairwise similarity	Pearson / Spearman	Estimate linear or monotonic co-abundance using jointly observed patients.	Similarity matrices.
Pairwise similarity	Overlap-adjusted correlation	Penalize scores estimated from few shared observations.	Reliability-adjusted similarities.
Pairwise similarity	Detection Jaccard	Capture shared protein quantification patterns across patients.	Missingness-aware similarity.
Pairwise similarity	Combined Pearson/Jaccard	Integrate abundance co-variation and co-detection.	Composite similarity matrix.
Pairwise similarity	RBF kernel	Measure nonlinear closeness of standardized protein profiles.	Kernel similarity matrix.
Pairwise similarity	Composite RBF/Jaccard	Combine nonlinear abundance similarity with co-detection.	Composite kernel matrix.
Representation learning	Mask-aware autoencoder	Learn embeddings by masked abundance reconstruction.	Protein embeddings and cosine similarities.
Representation learning	Patient-token transformer	Learn embeddings through attention over patient-level tokens.	Protein embeddings and cosine similarities.

Table 1: Summary of methods used for similarity calculation.

Evaluation: Pairwise Evaluation with CORUM

To evaluate whether the predicted similarities recovered known protein-complex relationships, I used CORUM as a benchmark. Each CORUM complex was expanded into all pairwise combinations of its subunits. Protein pairs from the same CORUM complex were treated as known co-complex positives. Evaluation was restricted to proteins present in both the proteomics dataset and CORUM. The metrics I chose are **AUROC**, **AUPRC**, **precision at K** , **recall at K** , and **enrichment at K** .

Stage 2: Graph Construction and Module Detection

After computing pairwise protein similarity matrices, I converted each method into a sparse weighted protein association network. In this graph, proteins were represented as nodes, retained predicted associations were represented as edges,

and similarity scores were used as edge weights.

I tested three graph construction strategies. The first retained the top 100,000 highest-scoring edges globally. The second retained edges above a fixed score threshold. The third retained the top 10 highest-scoring neighbors for each protein. The top-10-per-node graph was used as the main graph for downstream module annotation because it gives each protein a local neighborhood and reduces domination by globally high-scoring hub proteins.

From each weighted graph, I then applied two complementary module detection methods:

Leiden community detection [7] Leiden detects communities by optimizing a graph partition. It proceeds through three repeated operations. First, each node is locally moved to neighboring communities if this improves the weighted community quality. Second, the provisional communities are refined so that the nodes inside each community are well connected to one another, rather than being grouped only through weak bridges. Third, each refined community is aggregated into a super-node, and the same procedure is run again on the smaller graph. This iterative local-moving, refinement, and aggregation process produces a global partition of the network. I used Leiden to identify broad protein association modules, which may represent co-abundance programs or pathway-level communities rather than single physical complexes.

MCODE-style dense-core detection [1] MCODE searches for locally dense graph regions rather than partitioning the whole graph. It first computes a vertex score for each protein by examining its closed neighborhood, meaning the protein itself plus its direct neighbors. Within this neighborhood, the algorithm identifies a high-connectivity core and combines the local core level with the local density to score how suitable the protein is as a cluster seed. Proteins with high scores are used as seeds. Around each seed, neighboring proteins with scores close to the seed score are added to form a candidate cluster. The candidate cluster is then cleaned by removing weakly connected peripheral nodes and applying a k-core filter, so that the retained members form a dense local core. I used these MCODE cores as candidate complex-like subnetworks, because they emphasize local edge density and are therefore more specific than broad Leiden communities.

Together, Leiden and MCODE provide two levels of network organization: Leiden captures broad functional modules, while MCODE extracts dense local connections.

Key Results

CORUM evaluation showed that the missingness-aware and kernel-based methods performed best overall (Table 2). The composite RBF/Jaccard method achieved the highest AUPRC (0.1383), closely followed by intensity RBF (0.1382). The autoencoder cosine network was competitive, with AUPRC 0.1250 and Precision@10,000 of 0.3978, suggesting that learned embeddings captured useful structure beyond simple correlation. The transformer embedding was weaker in this run, with AUPRC 0.0744, possibly because the model capacity or training objective was not yet optimal for this dataset.

Method	AUROC	AUPRC	Prec.@1k	Enrich.@1k
Composite RBF + Jaccard	0.8012	0.1383	0.890	143.6
Intensity RBF	0.8022	0.1382	0.890	143.6
Combined Pearson + Jaccard	0.8321	0.1336	0.661	106.6
Pearson adjusted	0.8309	0.1333	0.661	106.6
Combined Spearman + Jaccard	0.8330	0.1261	0.587	94.7
Autoencoder cosine	0.8236	0.1250	0.584	98.5
Transformer cosine	0.7958	0.0744	0.404	68.1

Table 2: CORUM pair-level evaluation for representative PPI scoring methods.

Rather than focusing only on individual example modules, I also compared the module structure produced by different network construction methods. The top-10-per-node graphs produced broadly similar but not identical module decompositions (Table 3). The composite RBF/Jaccard graph produced the largest number of edges and the largest Leiden module, suggesting that this method captures broad abundance/missingness-driven biological programs. In contrast, the autoencoder cosine graph retained all 12,142 proteins but produced a slightly sparser graph, suggesting a more selective embedding-derived neighborhood structure.

The biological content of the modules was coherent (Table 4). In the autoencoder cosine network, the top Leiden modules corresponded to recognizable biological systems: TNFR1/signaling, immune/NADPH oxidase, endoplasmic reticulum glycoprotein processing, spliceosome/nuclear RNA processing, and mitochondrial oxidative phosphorylation. Its top MCODE core contained proteasome regulatory particle proteins, including PSMD11, PSMD6, PSMC5, PSMD8, PSMD13, PSMD12, PSMD3, PSMC2, PSMD14, and PSMD2, and matched the PA700 complex in CORUM.

Method	Edges	Leiden modules	Largest module	MCODE Modules	Top MCODE size
Autoencoder cosine	93,250	47	664	976	15
Composite RBF + Jaccard	107,520	36	1,102	1,126	25
Combined Pearson + Jaccard	101,991	39	780	935	23

Table 3: Comparison of module structure for the three methods selected for detailed annotation.

Method	Cluster	Size	Top CORUM match
Autoencoder cosine	Leiden L0001	664	TNFR1 signaling complex, TNF-induced
Autoencoder cosine	MCODE MC0001	15	PA700 complex
Composite RBF + Jaccard	Leiden L0002	694	cBAF complex
Combined Pearson + Jaccard	Leiden L0001	780	DNAJB11-SDF2L1 complex
Combined Pearson + Jaccard	Leiden L0002	582	F1F0-ATPase, mitochondrial

Table 4: Representative annotated modules recovered from the network analysis.

Functional Annotation Plan and Worked Examples

Functional annotation was used to assign biological meaning to the selected Leiden modules and MCODE dense cores. I focused on the top-10-per-node graphs for three representative similarity methods: autoencoder cosine similarity, intensity RBF similarity, and combined Pearson/Jaccard similarity. For each method, I annotated the top five Leiden modules and top five MCODE cores. Full module or core membership was used for enrichment analysis, while key proteins were used only for descriptive summaries.

The annotation pipeline used three layers of evidence. First, I tested each module or core for enrichment against known CORUM complexes using a hypergeometric test with Benjamini–Hochberg FDR correction. This step identified modules that may correspond to known protein complexes or sub-complexes.

Second, I used g:Profiler [5] to obtain broader annotations from GO Biological Process, GO Cellular Component, GO Molecular Function, Reactome, KEGG, WikiPathways, and CORUM.

Third, I used an LLM only as an interpretation layer. The LLM received structured evidence, including module size, density, key proteins, top enriched terms, adjusted p -values, and overlap genes, and produced concise biological explanations without generating new statistical evidence.

The functional annotation workflow was implemented in the repository as a set of reproducible worked examples. The first two evidence-generation layers, CORUM enrichment and g:Profiler annotation, are implemented in the notebooks under `notebooks/functional_annotation`. The final LLM-assisted interpretation outputs are stored in `outputs/network/annotation`.

Limitations and Future Work

I initially considered using part of CORUM for weakly supervised learning, where known co-complex pairs would serve as positive labels for fine-tuning the autoencoder or transformer embeddings. I did not implement this within the current timeframe because CORUM provides positive labels but not reliable negative pairs. Treating all unannotated pairs as negatives would be problematic, since many true interactions may simply be absent from CORUM. In addition, using CORUM for both training and evaluation would introduce a risk of data leakage and could inflate performance estimates.

With more time, I would construct a careful train–test split at the complex or protein level, so that held-out CORUM complexes could provide a cleaner evaluation set. The training portion could then be used for weakly supervised fine-tuning, positive-unlabeled learning, or contrastive learning with carefully sampled uncertain negatives.

More broadly, the current similarity-based pipeline relies on a guilt-by-association assumption: proteins with similar abundance or detection profiles are more likely to be functionally related or physically associated. This assumption is useful for recovering stable complexes, which explains the strong CORUM enrichment, but it may not generalize to all physical PPIs, especially transient or context-specific interactions. Future work could explore graph structure-learning methods such as SLAPS[4] or IDGL [3], where the adjacency matrix is learned jointly with the model rather than fixed from a predefined similarity score.

As mentioned in the interview, I could also combine more protein-level information. On top of patient profile similarities, I can also embed sequence similarities for modelling, which is also frequent in many similarities driven interaction prediction [8, 6].

References

- [1] Gary D. Bader and Christopher WV Hogue. An automated method for finding molecular complexes in large protein interaction networks. *BMC Bioinformatics*, 4(1):2, January 2003.
- [2] Roman Bushuiev, Anton Bushuiev, Raman Samusevich, Corinna Brungs, Josef Sivic, and Tomáš Pluskal. Self-supervised learning of molecular representations from millions of tandem mass spectra using DreaMS. *Nature Biotechnology*, 44(4):630–640, April 2026.
- [3] Yu Chen, Lingfei Wu, and Mohammed J. Zaki. Iterative Deep Graph Learning for Graph Neural Networks: Better and Robust Node Embeddings, October 2020. arXiv:2006.13009 [cs.LG].
- [4] Bahare Fatemi, Layla El Asri, and Seyed Mehran Kazemi. SLAPS: Self-Supervision Improves Structure Learning for Graph Neural Networks, October 2021. arXiv:2102.05034 [cs.LG].
- [5] Uku Raudvere, Liis Kolberg, Ivan Kuzmin, Tambet Arak, Priit Adler, Hedi Peterson, and Jaak Vilo. g:Profiler: a web server for functional enrichment analysis and conversions of gene lists (2019 update). *Nucleic Acids Research*, 47(W1):W191–W198, July 2019.
- [6] Andreas Schmidt, Ignasi Forne, and Axel Imhof. Bioinformatic analysis of proteomics data. *BMC Systems Biology*, 8(S2):S3, March 2014.
- [7] V. A. Traag, L. Waltman, and N. J. van Eck. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, March 2019.
- [8] Nazar Zaki, Sanja Lazarova-Molnar, Wassim El-Hajj, and Piers Campbell. Protein-protein interaction based on pairwise similarity. *BMC Bioinformatics*, 10:150, May 2009.

Appendix

A Model Selection

Baseline correlations. I computed Pearson and Spearman correlations using pairwise-complete observations, so each protein pair was compared only across patients where both proteins were quantified. To account for the reliability of each estimate, I saved the number of jointly observed patients and constructed overlap-adjusted correlations by downweighting correlations based on small patient overlap. In addition, I modeled missingness directly by computing Jaccard similarity between binary detection profiles. Finally, I combined abundance co-variation and detection-pattern similarity into a missingness-aware score.

Intensity RBF kernel. I implemented an intensity RBF kernel over protein-wise standardized abundance profiles. Unlike correlation, which measures whether two profiles move in the same direction, the RBF kernel measures whether two proteins are locally close in patient-profile space. This can capture nonlinear similarity and profile-level closeness that may be missed by correlation. A composite RBF/Jaccard kernel was also tested to combine continuous abundance-profile similarity with binary detection-pattern similarity.

Self-supervised autoencoder embedding. Due to a lack of reliable interaction labels for all proteins in our dataset, the self-supervised learning is a good approach. I trained a mask-aware autoencoder to learn protein embeddings from incomplete abundance profiles. A similar work was done before [2].

The model received the standardized protein abundance profile together with a binary observation mask. During training, some originally observed values were artificially hidden, and the model was trained to reconstruct only these artificially masked values. Naturally missing values were excluded from the reconstruction loss. Protein-pair similarity was then computed using cosine similarity between learned latent embeddings.

Patient-token transformer embedding. For self-supervised learning, I also trained a patient-token transformer that treats each patient measurement as a token containing the abundance value and observation mask. Self-attention was used to model dependencies among patient-level feature positions within each protein profile. The final transformer representation was used as a protein embedding, and protein-pair similarity was computed using cosine similarity.

B Mathematical Details

B.1 Pairwise-complete Correlation

For protein pair (i, j) , the jointly observed patient set is

$$\Omega_{ij} = \{n : X_{in} \text{ is observed and } X_{jn} \text{ is observed}\}.$$

The number of shared observations is

$$m_{ij} = |\Omega_{ij}|.$$

The NaN-aware Pearson correlation is

$$r_{ij} = \frac{\sum_{n \in \Omega_{ij}} (X_{in} - \bar{X}_{i, \Omega_{ij}})(X_{jn} - \bar{X}_{j, \Omega_{ij}})}{\sqrt{\sum_{n \in \Omega_{ij}} (X_{in} - \bar{X}_{i, \Omega_{ij}})^2} \sqrt{\sum_{n \in \Omega_{ij}} (X_{jn} - \bar{X}_{j, \Omega_{ij}})^2}}.$$

The overlap-adjusted correlation is

$$\tilde{r}_{ij} = r_{ij} \sqrt{\frac{m_{ij}}{N}}.$$

B.2 Detection-pattern Jaccard Similarity

For each protein i , the detection vector is

$$d_i \in \{0, 1\}^N,$$

where $d_{in} = 1$ if protein i was quantified in patient n .

The detection set is

$$D_i = \{n : d_{in} = 1\}.$$

The Jaccard similarity is

$$J_{ij} = \frac{|D_i \cap D_j|}{|D_i \cup D_j|}.$$

The combined correlation-detection score is

$$S_{ij}^{\text{corr+det}} = \alpha \tilde{r}_{ij} + (1 - \alpha) J_{ij}.$$

B.3 Intensity RBF Kernel

Each protein profile was standardized across observed patient values:

$$Z_{in} = \frac{X_{in} - \mu_i}{\sigma_i}.$$

The NaN-aware squared distance is

$$d_{ij}^2 = \frac{1}{m_{ij}} \sum_{n \in \Omega_{ij}} (Z_{in} - Z_{jn})^2.$$

The RBF kernel is

$$K_{ij}^{\text{RBF}} = \exp(-\gamma d_{ij}^2).$$

The composite RBF-detection kernel is

$$K_{ij}^{\text{comp}} = \alpha K_{ij}^{\text{RBF}} + (1 - \alpha) J_{ij}.$$

B.4 Self-supervised Reconstruction Loss

For both the autoencoder and transformer models, each protein profile was treated as one training sample. Let $Z_i \in R^N$ denote the protein-wise standardized abundance profile for protein i , and let $d_i \in \{0, 1\}^N$ denote its observation mask, where $d_{in} = 1$ if protein i was quantified in patient n . Natural missing values were filled with zero only after standardization, and the observation mask was provided to the model so that true missingness could be distinguished from observed abundance values close to zero.

During self-supervised training, a random subset of originally observed entries was artificially masked. Let

$$M_i \subseteq \{n : d_{in} = 1\}$$

denote the set of artificially masked patient positions for protein i . The corrupted input profile $\tilde{Z}_i$ was defined by replacing these selected observed values with zero:

$$\tilde{Z}_{in} = \begin{cases} 0, & n \in M_i, \\ Z_{in}, & n \notin M_i. \end{cases}$$

The model received the corrupted abundance profile and the observation mask as input. For the autoencoder, the input was the concatenated vector

$$h_i = [\tilde{Z}_i, d_i].$$

For the transformer, each patient position was represented as a token containing the corrupted abundance value and the observation indicator:

$$t_{in} = [\tilde{Z}_{in}, d_{in}].$$

The model produced a reconstruction $\hat{Z}_i$. The loss was computed only on artificially masked observed entries:

$$\mathcal{L}_{\text{rec}}(i) = \frac{1}{|M_i|} \sum_{n \in M_i} (\hat{Z}_{in} - Z_{in})^2.$$

Natural missing values were excluded from the loss because their true abundance values are unknown. This makes the task a masked reconstruction problem rather than an imputation task over all missing entries.

After training, the encoder output was used as a protein embedding. If u_i and u_j are the learned embeddings for proteins i and j , their embedding-based similarity was computed using cosine similarity:

$$S_{ij}^{\text{emb}} = \frac{u_i^\top u_j}{\|u_i\|_2 \|u_j\|_2}.$$

B.5 Patient-token Transformer Self-attention

In the transformer model, each protein profile was represented as a sequence of patient-level tokens. For protein i , the token at patient position n was

$$t_{in} = [\tilde{Z}_{in}, d_{in}],$$

where $\tilde{Z}_{in}$ is the corrupted standardized abundance value and d_{in} is the observation mask. Each token was projected into a d_{model} -dimensional hidden representation:

$$e_{in} = W_{\text{in}} t_{in} + b_{\text{in}}.$$

A learnable patient-position embedding p_n was added so that the model could distinguish different patient coordinates:

$$h_{in}^{(0)} = e_{in} + p_n.$$

A learnable global token was prepended to the sequence to summarize the whole protein profile:

$$H_i^{(0)} = [h_{i,\text{cls}}^{(0)}, h_{i1}^{(0)}, \dots, h_{iN}^{(0)}].$$

Within each transformer layer, multi-head self-attention was applied across the patient-token sequence. For one attention head, given hidden representation H , the query, key, and value matrices were

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V.$$

The attention weights were computed by scaled dot-product attention:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right),$$

and the attended representation was

$$\text{Attention}(Q, K, V) = AV.$$

Thus, each patient token could aggregate information from other patient positions in the same protein profile. This allows the model to learn dependencies among patient-level abundance patterns rather than treating the 118 patient values as independent features.

After the final transformer layer, the global representation was mapped to a latent protein embedding:

$$u_i = W_{\text{lat}} h_{i,\text{cls}}^{(L)} + b_{\text{lat}}.$$

The reconstructed abundance values were decoded from the patient-token hidden states:

$$\hat{Z}_{in} = f_{\text{dec}}(h_{in}^{(L)}).$$

The transformer was trained with the same masked reconstruction loss as the autoencoder, using only artificially masked observed entries. After training, cosine similarity between latent embeddings was used as the transformer-based protein similarity score.

B.6 CORUM Pairwise Enrichment

Precision at K is

$$\text{Precision@K} = \frac{\text{number of CORUM-positive pairs in top } K}{K}.$$

The background positive rate is

$$\text{BPR} = \frac{\text{number of CORUM-positive pairs}}{\text{number of evaluated protein pairs}}.$$

Enrichment at K is

$$\text{Enrichment@K} = \frac{\text{Precision@K}}{\text{BPR}}.$$

B.7 Leiden Modularity

For a weighted graph $G = (V, E, w)$, Leiden modularity is

$$Q = \frac{1}{2m} \sum_{i,j} \left(w_{ij} - \gamma \frac{k_i k_j}{2m} \right) \mathbf{1}(c_i = c_j),$$

where

$$k_i = \sum_j w_{ij}$$

and

$$2m = \sum_i k_i.$$

B.8 MCODE-style Dense-core Scoring

For a node v , the closed neighborhood is

$$N[v] = \{v\} \cup \{u : (u, v) \in E\}.$$

For a subgraph $H = (V_H, E_H)$, density is

$$\rho(H) = \frac{2|E_H|}{|V_H|(|V_H| - 1)}.$$

The MCODE-style vertex score is

$$s(v) = k_{\max}(v)\rho(H_v).$$

The dense-core score is

$$\text{MCODE}(C) = |C|\rho(C).$$

B.9 CORUM Module Enrichment

For a detected module A and a CORUM complex B , let M be the number of background proteins, K be the number of CORUM-complex proteins in the background, n be the module size, and $x = |A \cap B|$ be the overlap size. The hypergeometric enrichment p -value is

$$p = \sum_{t=x}^{\min(K,n)} \frac{\binom{K}{t} \binom{M-K}{n-t}}{\binom{M}{n}}.$$

C Similarities Visualization

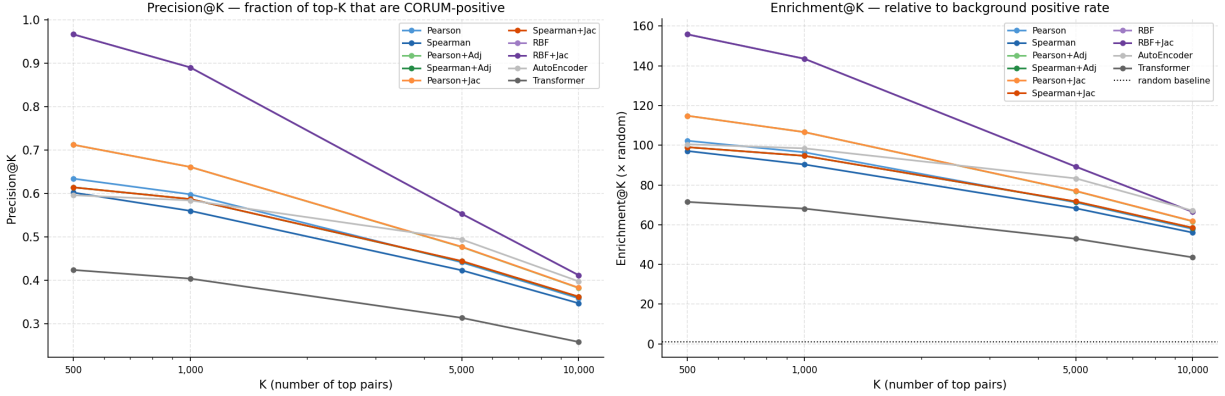


Figure 1: Precision and enrichment at top-ranked candidate PPI pairs. This plot shows how strongly each method prioritizes CORUM-supported interactions among the highest-ranked predictions.

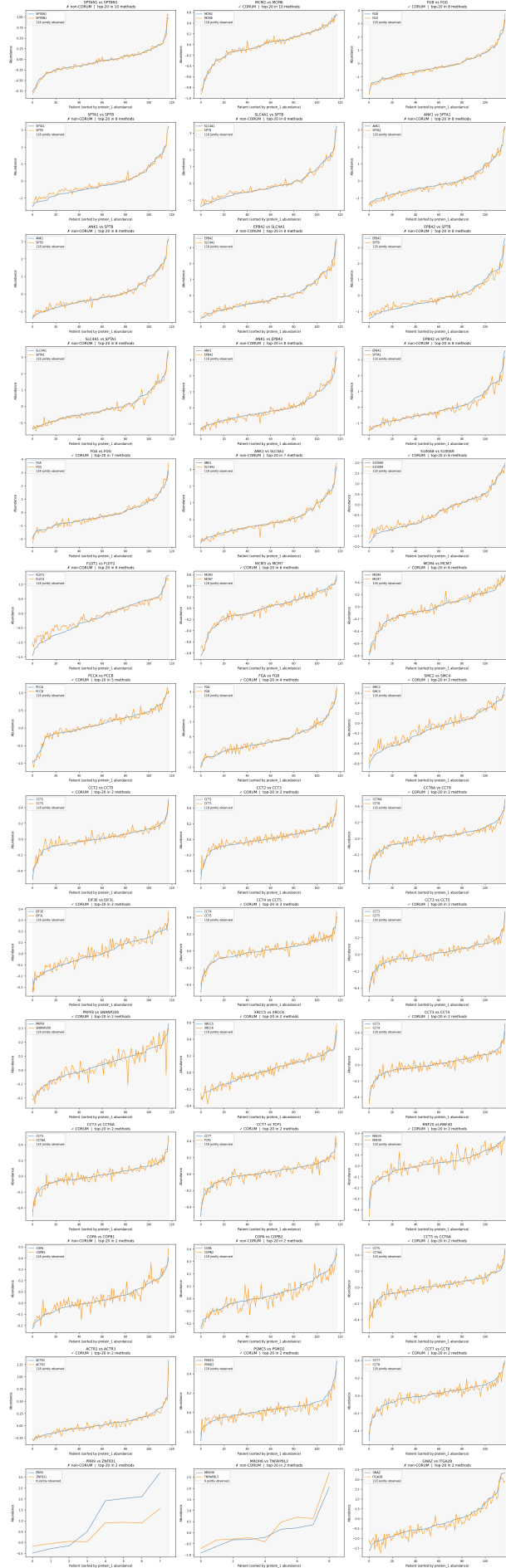


Figure 2: Abundance and observation-overlap profiles for representative top-ranked protein pairs.